

EXTEND WITH TOOLS

Subagents in the SDK

Define and invoke subagents to isolate context, run tasks in parallel, and apply specialized instructions in your Claude Agent SDK applications.

Subagents are separate agent instances that your main agent can spawn to handle focused subtasks. Use subagents to isolate context for focused subtasks, run multiple analyses in parallel, and apply specialized instructions without adding to the main agent's prompt.

This guide explains how to define and use subagents in the SDK using the `agents` parameter.

Overview

You can create subagents in three ways:

- **Programmatically:** use the `agents` parameter in your `query()` options ([TypeScript](#), [Python](#))
- **Filesystem-based:** define agents as markdown files in `.claude/agents/` directories (see [defining subagents as files](#))
- **Built-in general-purpose:** Claude can invoke the built-in `general-purpose` subagent at any time via the Agent tool without you defining anything

This guide focuses on the programmatic approach, which is recommended for SDK applications.

When you define subagents, Claude determines whether to invoke them based on each subagent's `description` field. Write clear descriptions that explain when the subagent should be used, and Claude will automatically delegate appropriate tasks. You can also explicitly request a subagent by name in your prompt (for example, "Use the code-reviewer agent to...").

Benefits of using subagents

Context isolation

Each subagent runs in its own fresh conversation. Intermediate tool calls and results stay inside the subagent; only its final message returns to the parent. See [What subagents inherit](#) for exactly what's

in the subagent's context.

Example: a `research-assistant` subagent can explore dozens of files without any of that content accumulating in the main conversation. The parent receives a concise summary, not every file the subagent read.

Parallelization

Multiple subagents can run concurrently, so independent subtasks finish in the time of the slowest one rather than the sum of all of them.

Example: during a code review, you can run `style-checker` , `security-scanner` , and `test-coverage` subagents simultaneously instead of sequentially.

Specialized instructions and knowledge

Each subagent can have tailored system prompts with specific expertise, best practices, and constraints.

Example: a `database-migration` subagent can have detailed knowledge about SQL best practices, rollback strategies, and data integrity checks that would be unnecessary noise in the main agent's instructions.

Tool restrictions

Subagents can be limited to specific tools, reducing the risk of unintended actions.

Example: a `doc-reviewer` subagent might only have access to Read and Grep tools, ensuring it can analyze but never accidentally modify your documentation files.

Creating subagents

Programmatic definition (recommended)

Define subagents directly in your code using the `agents` parameter. This example creates two subagents: a code reviewer with read-only access and a test runner that can execute commands. Claude invokes subagents through the `Agent` tool, so include `Agent` in `allowedTools` to auto-approve subagent invocations without a permission prompt.

Most examples on this page print only the final result. To confirm that Claude delegated to a subagent rather than answering directly, see [**Detecting subagent invocation.**](#)

```

import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions, AgentDefinition

async def main():
    async for message in query(
        prompt="Review the authentication module for security issues",
        options=ClaudeAgentOptions(
            # Auto-approve these tools, including Agent for subagent
            invocation
            allowed_tools=["Read", "Grep", "Glob", "Agent"],
            agents={
                "code-reviewer": AgentDefinition(
                    # description tells Claude when to use this subagent
                    description="Expert code review specialist. Use for
quality, security, and maintainability reviews.",
                    # prompt defines the subagent's behavior and expertise
                    prompt="""You are a code review specialist with expertise
in security, performance, and best practices.

```

When reviewing code:

- Identify security vulnerabilities
- Check for performance issues
- Verify adherence to coding standards
- Suggest specific improvements

Be thorough but concise in your feedback."""

```

        # tools restricts what the subagent can do (read-only
here)

        tools=["Read", "Grep", "Glob"],
        # model overrides the default model for this subagent
        model="sonnet",
    ),
    "test-runner": AgentDefinition(
        description="Runs and analyzes test suites. Use for test
execution and coverage analysis.",
        prompt="""You are a test execution specialist. Run tests
and provide clear analysis of results.

```

Focus on:

- Running test commands
- Analyzing test output
- Identifying failing tests
- Suggesting fixes for failures"""

```
        # Bash access lets this subagent run test commands
        tools=["Bash", "Read", "Grep"],
    ),
},
),
):
    if hasattr(message, "result"):
        print(message.result)

asyncio.run(main())
```

AgentDefinition configuration

Field	Type	Required	Description
description	string	Yes	Natural language description of when to use this agent
prompt	string	Yes	The agent's system prompt defining its role and behavior
tools	string[]	No	Array of allowed tool names. If omitted, inherits all tools
disallowedTools	string[]	No	Array of tool names to remove from the agent's tool set. MCP server-level patterns are also accepted: mcp__server or mcp__server__* removes every tool from that server, and mcp__* removes every MCP tool from any server
model	string	No	Model override for this agent. Accepts an alias such as 'fable' , 'opus' , 'sonnet' , 'haiku' , 'inherit' , or a full model ID. Defaults to main model if omitted
skills	string[]	No	List of skill names to preload into the agent's context at startup. Unlisted skills remain invocable through the Skill tool
memory	'user' 'project' 'local'	No	Memory source for this agent
mcpServers	(string object)[]	No	MCP servers available to this agent, by name or inline config
initialPrompt	string	No	Auto-submitted as the first user turn

Field	Type	Required	Description
			when this agent runs as the main thread agent. Ignored when the agent is invoked as a subagent
maxTurns	number	No	Maximum number of agentic turns before the agent stops
background	boolean	No	Run this agent as a non-blocking background task when invoked
effort	'low' 'medium' 'high' 'xhigh' 'max' number	No	Reasoning effort level for this agent
permissionMode	PermissionMode	No	Permission mode for tool execution within this agent

In the Python SDK, multi-word field names such as `disallowedTools` and `mcpServers` keep their camelCase spelling to match the wire format rather than following Python's `snake_case` convention. See the [AgentDefinition reference](#) for details.

❗ As of Claude Code v2.1.172, subagents can spawn their own subagents. A subagent five levels below the main agent cannot spawn further subagents, regardless of whether it runs in the foreground or background. To prevent a subagent from spawning others, omit `Agent` from its `tools` array or add it to `disallowedTools`. See [nested subagents](#) for the full depth rules.

Filesystem-based definition (alternative)

You can also define subagents as markdown files in `.claude/agents/` directories. See the [Claude Code subagents documentation](#) for details on this approach. Programmatically defined agents take precedence over filesystem-based agents with the same name.

❗ Even without defining custom subagents, Claude can spawn the built-in `general-purpose` subagent. This is useful for delegating research or exploration tasks without creating specialized agents. Include `Agent` in `allowedTools` so these invocations auto-approve without a permission prompt.

What subagents inherit

A subagent's context window starts fresh (no parent conversation) but isn't empty. The only channel from parent to subagent is the Agent tool's prompt string, so include any file paths, error messages, or decisions the subagent needs directly in that prompt.

The subagent receives	The subagent does not receive
Its own system prompt (<code>AgentDefinition.prompt</code>) and the Agent tool's prompt	The parent's conversation history or tool results
Project CLAUDE.md (loaded via <code>settingSources</code>)	Preloaded skill content, unless listed in <code>AgentDefinition.skills</code>
Tool definitions (inherited from parent, or the subset in <code>tools</code>)	The parent's system prompt

❗ The parent receives the subagent's final message verbatim as the Agent tool result, but may summarize it in its own response. To preserve subagent output verbatim in the user-facing response, include an instruction to do so in the prompt or `systemPrompt` option you pass to the `main query()` call.

Invoking subagents

Automatic invocation

Claude automatically decides when to invoke subagents based on the task and each subagent's `description` . For example, if you define a `performance-optimizer` subagent with the description “Performance optimization specialist for query tuning”, Claude will invoke it when your prompt mentions optimizing queries.

Write clear, specific descriptions so Claude can match tasks to the right subagent.

Explicit invocation

To guarantee Claude uses a specific subagent, mention it by name in your prompt:

```
"Use the code-reviewer agent to check the authentication module"
```

This bypasses automatic matching and directly invokes the named subagent.

Dynamic agent configuration

You can create agent definitions dynamically based on runtime conditions. This example creates a security reviewer with different strictness levels, using a more powerful model for strict reviews.

```

import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions, AgentDefinition

# Factory function that returns an AgentDefinition
# This pattern lets you customize agents based on runtime conditions
def create_security_agent(security_level: str) -> AgentDefinition:
    is_strict = security_level == "strict"
    return AgentDefinition(
        description="Security code reviewer",
        # Customize the prompt based on strictness level
        prompt=f"You are a {'strict' if is_strict else 'balanced'} security reviewer...",
        tools=["Read", "Grep", "Glob"],
        # Key insight: use a more capable model for high-stakes reviews
        model="opus" if is_strict else "sonnet",
    )

async def main():
    # The agent is created at query time, so each request can use different settings
    async for message in query(
        prompt="Review this PR for security issues",
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Grep", "Glob", "Agent"],
            agents={
                # Call the factory with your desired configuration
                "security-reviewer": create_security_agent("strict")
            },
        ),
    ):
        if hasattr(message, "result"):
            print(message.result)

asyncio.run(main())

```

Detecting subagent invocation

Subagents are invoked via the Agent tool. To detect when a subagent is invoked, check for

`tool_use` blocks where `name` is `"Agent"`. Messages from within a subagent's context include a

`parent_tool_use_id` field.

ⓘ The tool name was renamed from "Task" to "Agent" in Claude Code v2.1.63. Current SDK releases emit "Agent" in `tool_use` blocks but still use "Task" in the `system:init` tools list and in `result.permission_denials[].tool_name`. Checking both values in `block.name` ensures compatibility across SDK versions.

The message structure differs between SDKs. In Python, content blocks are accessed directly via `message.content`. In TypeScript, `SDKAssistantMessage` wraps the Claude API message, so content is accessed via `message.message.content`.

This example iterates through streamed messages, logging when a subagent is invoked and when subsequent messages originate from within that subagent's execution context.

```

import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions, AgentDefinition,
ToolUseBlock

async def main():
    async for message in query(
        prompt="Use the code-reviewer agent to review this codebase",
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Glob", "Grep", "Agent"],
            agents={
                "code-reviewer": AgentDefinition(
                    description="Expert code reviewer.",
                    prompt="Analyze code quality and suggest improvements.",
                    tools=["Read", "Glob", "Grep"],
                )
            },
        ),
    ):
        # Check for subagent invocation. Match both names: older SDK
        # versions emitted "Task", current versions emit "Agent".
        if hasattr(message, "content") and message.content:
            for block in message.content:
                if isinstance(block, ToolUseBlock) and block.name in (
                    "Task",
                    "Agent",
                ):
                    print(f"Subagent invoked:
{block.input.get('subagent_type')}")

            # Check if this message is from within a subagent's context
            if hasattr(message, "parent_tool_use_id") and
message.parent_tool_use_id:
                print(" (running inside subagent)")

            if hasattr(message, "result"):
                print(message.result)

asyncio.run(main())

```

Resuming subagents

Subagents can be resumed to continue where they left off. Resumed subagents retain their full conversation history, including all previous tool calls, results, and reasoning. The subagent picks up exactly where it stopped rather than starting fresh.

When a subagent completes, the Agent tool result includes a text block containing `agentId: <id>`. The built-in `Explore and Plan agents` are one-shot and do not return an `agentId`, so use a custom agent or `general-purpose` when you need to resume. To resume a subagent programmatically:

1. **Capture the session ID:** Extract `session_id` from messages during the first query
2. **Extract the agent ID:** Parse `agentId` from the Agent tool result text
3. **Resume the session:** Pass `resume: sessionId` in the second query's options, and include the agent ID in your prompt

❗ You must resume the same session to access the subagent's transcript. Each `query()` call starts a new session by default, so pass `resume: sessionId` to continue in the same session.

When using a custom agent, pass the same agent definition in the `agents` parameter for both queries.

The example below defines a custom `endpoint-finder` agent. The first query runs it and captures the session ID and agent ID from the Agent tool result, then the second query resumes the session to ask a follow-up question that requires context from the first analysis.

```

import asyncio
import re
from claude_agent_sdk import query, ClaudeAgentOptions, AgentDefinition,
ToolResultBlock

AGENTS = {
    "endpoint-finder": AgentDefinition(
        description="Locates and catalogs API endpoints in a codebase.",
        prompt="You find and document API endpoints. Report each endpoint's
path, method, and handler.",
        tools=["Read", "Grep", "Glob"],
    )
}

def extract_agent_id(block: ToolResultBlock) -> str | None:
    """Extract agentId from an Agent tool result's text content."""
    parts = block.content if isinstance(block.content, list) else [{"text":
block.content}]
    for part in parts:
        if match := re.search(r"agentId:\s*([\w-]+)", part.get("text") or ""):
            return match.group(1)
    return None

async def main():
    agent_id = None
    session_id = None

    # First invocation - run the endpoint-finder subagent
    try:
        async for message in query(
            prompt="Use the endpoint-finder agent to find all API endpoints in
this codebase",
            options=ClaudeAgentOptions(allowed_tools=["Read", "Grep", "Glob",
"Agent"], agents=AGENTS),
        ):
            # Capture session_id from ResultMessage (needed to resume this
session)

            if hasattr(message, "session_id"):
                session_id = message.session_id
            # Search tool results for the agentId trailer
            for block in getattr(message, "content", None) or []:
                if isinstance(block, ToolResultBlock):

```

```

        agent_id = extract_agent_id(block) or agent_id
    # Print the final result
    if hasattr(message, "result"):
        print(message.result)
except Exception as error:
    # A single-shot query() raises after yielding an error result,
    # so session_id and agent_id have already been captured by the loop
    above.
    print(f"Session ended with an error: {error}")

# Second invocation - resume and ask follow-up
if agent_id and session_id:
    async for message in query(
        prompt=f"Resume agent {agent_id} and list the top 3 most complex
endpoints",
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Grep", "Glob", "Agent"],
agents=AGENTS, resume=session_id
        ),
    ):
        if hasattr(message, "result"):
            print(message.result)
else:
    print("No agentId found in the first query, so there is no subagent to
resume.")

asyncio.run(main())

```

Subagent transcripts persist independently of the main conversation:

- **Main conversation compaction:** When the main conversation compacts, subagent transcripts are unaffected. They're stored in separate files.
- **Session persistence:** Subagent transcripts persist within their session. You can resume a subagent after restarting Claude Code by resuming the same session.
- **Automatic cleanup:** Transcripts are cleaned up based on the `cleanupPeriodDays` setting (default: 30 days).

Tool restrictions

Subagents can have restricted tool access via the `tools` field:

- **Omit the field:** agent inherits all available tools (default)
- **Specify tools:** agent can only use listed tools

This example creates a read-only analysis agent that can examine code but cannot modify files or run commands.

```
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions, AgentDefinition

async def main():
    async for message in query(
        prompt="Analyze the architecture of this codebase",
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Grep", "Glob", "Agent"],
            agents={
                "code-analyzer": AgentDefinition(
                    description="Static code analysis and architecture
review",
                    prompt="""You are a code architecture analyst. Analyze
code structure,
identify patterns, and suggest improvements without making changes.""",
                    # Read-only tools: no Edit, Write, or Bash access
                    tools=["Read", "Grep", "Glob"],
                )
            },
        ),
    ):
        if hasattr(message, "result"):
            print(message.result)

asyncio.run(main())
```

Common tool combinations

Use case	Tools	Description
Read-only analysis	Read , Grep , Glob	Can examine code but not modify or execute
Test execution	Bash , Read , Grep	Can run commands and analyze output

Use case	Tools	Description
Code modification	<code>Read</code> , <code>Edit</code> , <code>Write</code> , <code>Grep</code> , <code>Glob</code>	Full read/write access without command execution
Full access	All tools	Inherits all tools from parent (omit <code>tools</code> field)

Scale up with dynamic workflows

Subagents work well for a few delegated tasks per turn. For runs that coordinate dozens to hundreds of agents, use the `Workflow` tool, which moves the orchestration into a script the runtime executes outside the conversation context. See [dynamic workflows](#) for how workflows differ from turn-by-turn subagent delegation.

The `Workflow` tool is available in the TypeScript Agent SDK v0.3.149 and later. Include `Workflow` in `allowedTools` to auto-approve workflow runs. The tool input and output schemas are listed in the [TypeScript reference](#).

Troubleshooting

Claude not delegating to subagents

If Claude completes tasks directly instead of delegating to your subagent:

- Check Agent invocations are approved:** include `Agent` in `allowedTools` to auto-approve subagent calls. Without it, Agent invocations fall through to your `canUseTool` callback or, in `dontAsk` mode, are denied
- Use explicit prompting:** mention the subagent by name in your prompt (for example, “Use the code-reviewer agent to...”)
- Write a clear description:** explain exactly when the subagent should be used so Claude can match tasks appropriately

Filesystem-based agents not loading

Agents defined in `.claude/agents/` are loaded at startup only. If you create a new agent file while Claude Code is running, restart the session to load it.

Windows: long prompt failures

On Windows, subagents with very long prompts may fail due to command line length limits (8191 chars). Keep prompts concise or use filesystem-based agents for complex instructions.

Related documentation

- **Claude Code subagents**: comprehensive subagent documentation including filesystem-based definitions
- **Dynamic workflows**: orchestrate many subagents from a script for jobs too large for one conversation
- **SDK overview**: getting started with the Claude Agent SDK